Software Design Patterns Research (SynChef)

1. Factory Method

- Category: Creational
- Problem It Solves: Object creation logic gets duplicated and scattered, causing inconsistent objects and hard-to-maintain constructors.
- How It Works: A dedicated factory method/class centralizes object creation and hides construction details.
- Real-World Example: Backend notification system creates different notification objects (welcome, comment, reminder) with consistent defaults.
- Use Case in SynChef: `NotificationFactory` now creates `AppNotification` instances for welcome and comment events.

2. Builder

- Category: Creational
- Problem It Solves: Entities with many optional fields become hard to construct safely using long constructors or multiple setters.
- How It Works: Builder exposes step-by-step fluent setters and builds a final immutable-like configured object.
- Real-World Example: Building API payloads or domain entities with optional metadata and flags.
- Use Case in SynChef: `AppNotification` now supports Lombok `@Builder` to create complete notifications clearly and safely.

3. Facade

- Category: Structural
- Problem It Solves: Controllers/clients directly depend on many subsystems, increasing coupling and making orchestration repetitive.
- How It Works: A facade provides a simplified API over multiple services.
- Real-World Example: CheckoutFacade in e-commerce coordinating cart, payment, inventory, and shipping services.
- Use Case in SynChef: `RecipePreparationFacade` wraps recipe scaling and timer orchestration so the controller has one integration point.

4. Adapter

- Category: Structural
- Problem It Solves: Existing interfaces are incompatible across modules/vendors, blocking integration.
- How It Works: Adapter converts one interface into another expected by clients.
- Real-World Example: Mobile app adapter converting internal timer state to websocket event format.
- Use Case in SynChef: Potential use for integrating external nutrition APIs into existing `Recipe` DTOs without modifying core domain models.

5. Strategy

- Category: Behavioral
- Problem It Solves: Multiple algorithms for the same behavior are hard-coded via conditionals, making extension risky.
- How It Works: Encapsulate each algorithm into strategy classes and inject/select them at runtime.
- Real-World Example: Pricing strategy (regular, discount, promo) in commerce systems.
- Use Case in SynChef: Ingredient rounding and timer scaling algorithms were extracted into strategy interfaces and implementations.

6. Observer

- Category: Behavioral
- Problem It Solves: Many components need to react to state changes without tight coupling.
- How It Works: Subject publishes events; observers subscribe and react asynchronously.
- Real-World Example: Real-time stock ticker or chat updates via websocket topics.
- Use Case in SynChef: Existing websocket topic `/topic/timer-updates` follows observer-style pub/sub for timer events.

Summary

SynChef naturally benefits from combining:

- Creational patterns for safe object construction (`Factory Method`, `Builder`)
- Structural patterns for cleaner architecture boundaries (`Facade`)
- Behavioral patterns for extensible algorithms and real-time interaction (`Strategy`, `Observer`)

Adapter is identified as a strong next pattern for future external API integration.